

Bagian 1

1. Praktik Bagian 1

1. Hashing sebelum file sample.txt diubah

```
(labibaadinda@vbox)-[/home/labibaadinda]
PS> Get-FileHash sample.txt -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	336F5841EDB4DDFD6D7D4FFE5CB23CAD43B1A6552FF5A96C655DC4F8CD822F1C	/home/labibaadinda/sample.txt

2. Hashing setelah file sample.txt diubah

```
(labibaadinda@vbox)-[/home/labibaadinda]
PS> nano sample.txt
(labibaadinda@vbox)-[/home/labibaadinda]
PS> Get-FileHash sample.txt -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	361CFAD9D6F7E9B4C4F9C160ECBE632C5A164F1CB5F9D6AE438EB48B5A7A91A0	/home/labibaadinda/sample.txt

2. Pertanyaan dan jawaban

1. Kenapa hash cocok untuk verifikasi integritas?

Karena data berubah satu bit aja, hashnya akan sangat berubah (ini disebut sebagai *avalanche effect*). Hash ini penting untuk verifikasi integritas data karena berfungsi sebagai “sidik jari digital” dari suatu data. Dengan menggunakan fungsi hash seperti SHA-256 atau MD5 (yang saya terapkan SHA-256), data apapun akan diubah menjadi nilai unik dengan panjang tetap. Keunggulan utamanya adalah sifat sensitif terhadap perubahan sekecil apapun (misal seperti yang saya sebutkan diawal satu bit data berubah, maka nilai hash yang dihasilkan akan berbeda secara signifikan. Karena hal tersebut memungkinkan untuk mendeteksi apakah suatu data tetap utuh atau sudah mengalami perubahan.

2. Dalam skenario kerja, kapan verifikasi *hash* wajib dilakukan?

Verifikasi hash menjadi wajib dilakukan ketika ada risiko perubahan data, manipulasi, atau ketika integritas data harus bisa dibuktikan. Salah satu skenario utamanya adalah saat menerima file dari luar sistem seperti *attachment* email, hasil *download*, atau file dari pihak ketiga. Disini, hash (misalnya dengan SHA-256) digunakan untuk memastikan file tidak disusupi malware atau berubah selama proses transfer, tanpa verifikasi file yang terlihat normal bisa jadi sudah dimodifikasi secara berbahaya.

Bagian 2

1. Praktik bagian 2

- a. Buat file laporan.txt dan menyisipkan stream tersembunyi

```
C:\Users\Asus>echo "file normal" > laporan.txt  
C:\Users\Asus>echo "data rahasia" > laporan.txt:rahasia.txt
```

- b. Isi file nya

```
PS C:\Users\Asus> Get-Content laporan.txt  
"file normal"  
PS C:\Users\Asus> Get-Content laporan.txt -Stream rahasia.txt  
"data rahasia"
```

2. Pertanyaan dan jawaban

1. Mengapa ADS sering disalahgunakan malware?

ADS pada NTFS sering disalahgunakan karena memungkinkan penyimpanan data tersembunyi tanpa terlihat oleh pengguna biasa maupun tool sederhana. Malware memanfaatkan ADS untuk menyimpan payload, script, atau konfigurasi berbahaya di dalam file yang tampak normal (misalnya laporan.txt). Karena Windows Explorer dan banyak proses inspeksi dasar tidak menampilkan stream tambahan, attacker bisa menyembunyikan aktivitasnya dengan lebih mudah. Selain itu, ADS tidak mengubah tampilan file utama secara signifikan, sehingga teknik ini efektif untuk *stealth* dan *persistence*. Pada beberapa kasus, malware bahkan dijalankan langsung dari ADS atau dipanggil oleh script tertentu, sehingga keberadaannya makin sulit dideteksi.

2. Apa risiko jika proses inspeksi file hanya melihat “nama file utama”

Jika inspeksi hanya berfokus pada file utama (*main stream*), maka semua data tersembunyi di ADS akan terlewat. Ini berisiko tinggi karena file yang terlihat aman sebenarnya bisa menyimpan malware, *credential*, atau data sensitif di dalam *stream* tersembunyi. Akibatnya, sistem keamanan bisa salah menganggap file tersebut tidak berbahaya (*false negative*). Dalam konteks incident response atau digital forensics, hal ini juga dapat menyebabkan analisis tidak lengkap karena sebagian bukti tersembunyi tidak terdeteksi. Oleh karena itu, inspeksi yang aman harus mencakup seluruh *stream* dalam file, bukan hanya nama atau isi utama saja.

Bagian 3

1. Praktik bagian 3

a. Membuat user baru

```
labibaadinda@labibaadinda:~$ sudo adduser labuser
[sudo: authenticate] Password:
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for labuser
Enter the new value, or press ENTER for the default
    Full Name []: Lab User
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
```

b. Cek isi /etc/passwd dan /etc/shadow

```
labibaadinda@labibaadinda:~$ grep '^labuser:' /etc/passwd
labuser:x:1001:1001:Lab User,,,:/home/labuser:/bin/bash
labibaadinda@labibaadinda:~$ grep '^labuser:' /etc/shadow
grep: /etc/shadow: Permission denied
labibaadinda@labibaadinda:~$ sudo grep '^labuser:' /etc/shadow
labuser:$y$j9T$E5XY60Tcb9/ydJFZV.rGW/$iQGQyB3I08TQ37uFYq6ETv4IfvybZhVKsFhfJFHq6:20557:0:99999:7:::
```

2. Pertanyaan dan jawaban

a. Mengapa /etc/shadow harus hanya bisa diakses root?

Karena file /etc/shadow menyimpan informasi autentikasi paling sensitif di sistem Linux, yaitu hash password dari semua user. Jika file ini bisa diakses oleh user biasa, maka keamanan seluruh file bisa terancam. Meskipun dalam bentuk hash bukan plaintext, hash tetap bisa diserang menggunakan teknik seperti *brute force* atau *dictionary attack*. Jika attacker mendapatkan isi file ini, mereka bisa mencoba memecahkan password secara offline tanpa batasan, yang mana itu jauh lebih berbahaya dibandingkan serangan login biasa. Dengan membatasi akses hanya ke root, sistem menerapkan prinsip *least privilege*, yaitu hanya pihak yang benar-benar membutuhkan yang bisa mengakses data sensitif. Selain itu juga semisal /etc/shadow bisa dimodifikasi oleh sembarang user, attacker dapat langsung mengganti hash password dan mengambil alih akun lain, termasuk akun root.

b. Apa perbedaan peran /etc/passwd vs /etc/shadow?

/etc/passwd menyimpan informasi dasar user seperti username, UID, home directory, dan shell, serta bisa dibaca oleh semua user karena tidak berisi data sensitif. Sementara itu, /etc/shadow menyimpan hash password dan informasi keamanan terkait, biasanya menggunakan algoritma seperti SHA-512, dan hanya bisa diakses oleh root. Intinya, /etc/passwd berisi identitas user (publik), sedangkan /etc/shadow berisi autentikasi (rahasia).

Bagian 4

1. Praktik bagian 4

a. Identifikasi salt

```
labibaadinda@labibaadinda:~$ sudo grep '^labuser:' /etc/shadow | cut -d ':' -f2  
$y$j9T$E5XY60Tcb9/ydJFZV.rGW/$iQGQyB3IO8TQ37uFYq6ETv4IfvybZhVKsFhfJFHq6
```

Format secara umum :

\$id\$salt\$hash

- id = y (algoritma yescrypt)

- salt = j9T\$E5XY60Tcb9/ydJFZV.rGW

j9T → parameter cost / konfigurasi yescrypt

E5XY60Tcb9/ydJFZV.rGW → nilai salt random

- digest/hash = iQGQyB3IO8TQ37uFYq6ETv4IfvybZhVKsFhfJFHq6

nah digest/hash tersebut itu adalah hasil dari password dan salt yang diproses dengan algoritma yescrypt

b. Legacy tanpa salt

```
labibaadinda@labibaadinda:~$ printf "123456  
password  
admin  
qwerty  
" > weakpass.txt  
labibaadinda@labibaadinda:~$ while read -r p; do printf "%s" "$p" | md5sum; done < weakpass.txt > md5_table.txt
```

Table hash dapat digunakan berulang kali untuk menyerang banyak sistem/user

```
labibaadinda@labibaadinda:~$ cat md5_table.txt  
e10adc3949ba59abbe56e057f20f883e -  
5f4dcc3b5aa765d61d8327deb882cf99 -  
21232f297a57a5a743894a0e4a801fc3 -  
d8578edf8458ce06fbc5bb76a58c5ca4 -
```

2. Pertanyaan dan jawaban

1. Mengapa salt mencegah penggunaan ulang tabel hash (rainbow/precompute)?

Salt mencegah penggunaan ulang tabel hash (rainbow/precompute) karena membuat setiap proses hashing menjadi unik untuk setiap user atau setiap instance password. Pada sistem tanpa salt, satu password selalu menghasilkan hash yang sama (misalnya dengan MD5), sehingga attacker bisa membuat tabel hash sekali lalu menggunakannya berulang kali untuk banyak target.

Namun ketika salt ditambahkan, password tidak lagi di-hash secara langsung, melainkan dikombinasikan dengan nilai acak (salt) sebelum di-hash. Akibatnya, password yang sama akan menghasilkan hash yang berbeda jika salt-nya berbeda. Karena setiap user memiliki salt yang unik, maka tabel hash yang dibuat

sebelumnya tidak bisa digunakan kembali. Attacker harus membuat tabel baru untuk setiap kombinasi salt, yang secara praktis sangat tidak efisien dan memakan sumber daya besar. Hal tersebut yang membuat salt efektif dalam mencegah serangan berbasis precomputation seperti rainbow table.

2. Jika dua pengguna memakai password yang sama, apa yang terjadi pada hash salted?

Hash yang dihasilkan tetap berbeda. Hal ini karena setiap user memiliki nilai salt yang unik, sehingga proses hashing menjadi berbeda meskipun input passwordnya sama.

Secara konsep:

User A : password + salt A = hash A

User B : password + salt B = hash B

Walaupun password User A dan User B identik, hasil akhirnya (digest) tidak akan sama karena dipengaruhi oleh salt yang berbeda. Ini berbeda dengan sistem legacy tanpa salt, di mana password yang sama akan menghasilkan hash yang sama.

Bagian 5

1. Praktik bagian 5

```
labibaadinda@labibaadinda:~$ time while read -r p; do printf "%s" "$p" | sha256sum >/dev/null; done < weakpass.txt
real    0m0.044s
user    0m0.019s
sys     0m0.028s
```

Berdasarkan hasil nya menunjukkan bahwa hash SHA-256 menghasilkan output dengan sangat cepat dan SHA-256 menunjukkan lebih cocok untuk cek integritas data bukan untuk password. Dengan kecepatan seperti ini, seorang attacker dapat mencoba sangat banyak kombinasi password dalam waktu singkat (brute force), sehingga meningkatkan peluang keberhasilan serangan.

2. Pertanyaan dan jawaban
 - a. Jelaskan perbedaan “lebih lambat karena algoritma” vs “lebih mahal karena salt tidak bisa digunakan ulang”.

“Lebih lambat karena algoritma” berarti setiap proses hashing memang sengaja dibuat berat oleh desain algoritma modern seperti bcrypt atau Argon2. Algoritma ini menggunakan teknik seperti key stretching (pengulangan proses hashing) dan penggunaan memori besar, sehingga satu kali percobaan password saja membutuhkan waktu lebih lama. Akibatnya, attacker tidak bisa mencoba banyak password dalam waktu singkat karena setiap percobaan “mahal” secara komputasi.

Sementara itu, “lebih mahal karena salt tidak bisa digunakan ulang” bukan tentang lambatnya satu proses hashing, tetapi tentang tidak bisa reuse hasil kerja sebelumnya. Tanpa salt, attacker bisa membuat tabel hash (precompute/rainbow table) sekali lalu menggunakannya untuk banyak user atau sistem. Dengan adanya salt, setiap user memiliki nilai berbeda, sehingga attacker harus menghitung ulang hash untuk setiap kombinasi password dan setiap user. Ini membuat total pekerjaan menjadi jauh lebih banyak, walaupun tiap hashing tidak selalu lebih lambat.

b. Kebijakan apa saja yang memperkecil peluang password berhasil ditebak?

Kebijakan yang dapat memperkecil peluang password berhasil ditebak berfokus pada membuat password lebih sulit ditebak dan membatasi percobaan attacker. Dengan menggunakan algoritma hashing modern seperti Argon2 atau bcrypt yang sudah dilengkapi salt dan cost factor, sehingga setiap percobaan password menjadi mahal secara komputasi. Selain itu, terapkan kebijakan password yang kuat, seperti panjang minimal (misalnya minimal 12 karakter), kombinasi huruf besar, kecil, angka, dan simbol, serta melarang penggunaan password umum atau yang mudah ditebak. Serta, penting untuk menerapkan mekanisme pembatasan percobaan login seperti rate limiting atau account lockout agar serangan brute force online tidak bisa dilakukan secara bebas. Penggunaan multi-factor authentication (MFA) juga sangat efektif karena menambahkan lapisan keamanan tambahan meskipun password berhasil ditebak. Di sisi lain, edukasi pengguna agar tidak menggunakan password yang sama di banyak layanan (password reuse) juga penting untuk mencegah efek domino jika terjadi kebocoran data. Terakhir, monitoring dan audit login secara berkala membantu mendeteksi aktivitas mencurigakan sejak dini sehingga risiko dapat diminimalkan.